



Project 1_ Realtime Sou...
PDF

Build the given project from scratch. I am a beginner in this field so guide me in each step. DO the porject step-by-step. Move to next step only when I tell. Also refer this Youtube video to build this project: <https://www.youtube.com/watch?v=eLy1JsBwYqQ>

Complete details and architecture is attached. Refer that in detail. So now start with step 1 and list all the steps to build this project. In each step explain why I am doing this and explanation behind that.

Great — this is an **excellent project choice** (and very aligned with AI Engineer roles). I'll guide you step-by-step like a mentor. We will **NOT rush** — you'll build real understanding.



Step 1: Understand the Project + Full Roadmap

Before writing a single line of code, we need clarity.



What are we building?

From your document:

A system that takes a GitHub repo → processes code → lets users ask questions about it using AI

Project 1_ Realtime Source Code...



Core Idea (Simplified)

You will build a system like this:

```
User → enters GitHub repo
      ↓
System → downloads code
      ↓
System → breaks code into chunks
      ↓
System → converts chunks → embeddings (numbers)
      ↓
```



```
Store → in vector database (ChromaDB)
      ↓
User asks question
      ↓
System → finds relevant code chunks
      ↓
LLM → generates answer
```

This is called:

👉 **RAG (Retrieval-Augmented Generation)**

🏗️ Architecture (From your PDF)

The *diagram on page 2* shows this flow clearly:

- GitHub Repo → GitPython (clone)
- Loader → read `.py` files
- Text Splitter → chunk code
- OpenAI → embeddings
- ChromaDB → store vectors
- Flask → UI + API
- Chatbot → answers questions

 [Project 1_ Realtime Source Code...](#)

🧭 FULL STEP-BY-STEP ROADMAP

We will follow this exact sequence (very important):

◆ PHASE 1: Foundations (Step 1–5)

✅ Step 1 (CURRENT STEP)

Understand architecture + tools

- Why: Without this, you'll just copy code blindly
-

Step 2: Environment Setup

- Install Python, VS Code
- Create virtual environment
- Install dependencies

👉 Why?

To isolate your project and avoid dependency issues

Step 3: Project Structure Setup

- Create folders like:

```
project/  
|  
├─ app/  
├─ ingestion/  
├─ chatbot/  
├─ vectorstore/  
└─ utils/
```



👉 Why?

Clean structure = scalable + professional

Step 4: Get API Keys

- OpenAI / Groq (as per your choice)

👉 Why?

Needed for embeddings + chat model

Step 5: Basic Flask App Setup

- Create simple server
- Test endpoint

👉 Why?

This will be your backend system

◆ PHASE 2: Core RAG Pipeline (Step 6–10)

Step 6: Clone GitHub Repo

- Use `GitPython`

👉 Why?

We need actual code to analyze

Step 7: Load Python Files

- Use LangChain loader

👉 Why?

Convert files → readable documents

Step 8: Text Chunking

- Split code into smaller parts

👉 Why?

LLMs cannot handle large files directly

Step 9: Create Embeddings

- Convert text → vectors

👉 Why?

This enables semantic search

Step 10: Store in ChromaDB

- Save embeddings

👉 Why?

So we can retrieve later

◆ PHASE 3: Chatbot System (Step 11–13)

Step 11: Retriever Setup

- Query vector DB

👉 Why?

Find relevant code snippets

Step 12: RAG Chain

- Combine:

- Retriever + LLM

👉 Why?

This is the brain of your system

Step 13: Conversational Memory

- Store previous chats

👉 Why?

Follow-up questions become possible

◆ PHASE 4: Frontend + Integration (Step 14–16)

Step 14: UI (Flask templates)

- Input repo URL
 - Chat interface
-

Step 15: API Routes

- `/ingest`
- `/chat`

👉 Matches diagram from PDF

Step 16: Connect Everything

- End-to-end pipeline
-

◆ PHASE 5: Advanced + Deployment (Step 17–20)

Step 17: Improve Chunking (Interview important)

- Keep functions + docstrings together
-

Step 18: Metadata Filtering

- Filter by file type
-

Step 19: Dockerization

- Make project deployable
-

Step 20: Deployment

- Render / AWS / Railway
-

Important Mindset

You are a beginner — so:

- Don't try to understand everything at once
 - Focus **step-by-step**
 - Ask doubts anytime
-

What we do NEXT

We move to:

Step 2: Environment Setup (hands-on coding)

I will guide you like this:

- exact commands
 - what each command does
 - common errors
-

Your Task Now

Before we proceed:

 Tell me:

- Are you using **Windows / Mac / Linux**?
- Do you have **Python installed?** (which version?)

- Are you using **VS Code**?

Once you confirm, we'll start **Step 2 (setup)** 🖥️

Move to Step 2

Perfect. Let's move into **Step 2: Environment Setup** — this is where we prepare your system to build the project properly.

⚙️ **STEP 2: Environment Setup (VERY IMPORTANT)**

This step might feel “basic”, but it's **critical**. Most beginners get stuck later because they skip or mess this up.

🎯 **Goal of This Step**

Set up a clean development environment so we can:

- Install required libraries (LangChain, Flask, ChromaDB, etc.)
- Avoid dependency conflicts
- Run the project smoothly

👉 This directly supports the “**Local Package Setup**” mentioned in your PDF under *Section 1*

📄 Project 1_ Realtime Source Code...

🧩 **Step 2.1: Check Python Installation**

✅ **Run this in terminal:**

❏ Bash



```
python --version
```

👉 **Expected output:**

```
Python 3.10+
```



❗ **If Python is NOT installed:**

Download from:

👉 <https://www.python.org/downloads/> ↗

⚠ **Important:**

Use **Python 3.10 or 3.11**

👉 NOT 3.13 (many AI libraries break)

🧩 Step 2.2: Create Project Folder

Run:

❏ Bash



```
mkdir source-code-analyzer  
cd source-code-analyzer
```

👉 **Why?**

We keep everything organized in one place.

🧩 Step 2.3: Create Virtual Environment

Run:

❏ Bash



```
python -m venv venv
```

👉 **What this does:**

Creates an isolated Python environment:

```
source-code-analyzer/  
└─ venv/
```



🧠 Why this is IMPORTANT:

Without this:

- Library versions conflict
- Projects break randomly
- Hard to debug

🧩 Step 2.4: Activate Virtual Environment

👉 Windows:

</> Bash



```
venv\Scripts\activate
```

👉 Mac/Linux:

</> Bash



```
source venv/bin/activate
```

✅ You should see:

(venv) C:\...



🧩 Step 2.5: Install Required Libraries

Now install all core dependencies:

</> Bash



```
pip install flask langchain openai chromadb gitpython tiktoken python-dotenv
```

🧠 What each library does:

Library	Why we need it
Flask	Web backend
LangChain	RAG pipeline
OpenAI	Embeddings + LLM
ChromaDB	Vector database
GitPython	Clone GitHub repos
tiktoken	Token handling
python-dotenv	Store API keys

Step 2.6: Save Dependencies

 Bash



```
pip freeze > requirements.txt
```

Why?

So your project can be recreated anywhere

Step 2.7: Open Project in VS Code

Steps:

1. Open VS Code
2. Click **Open Folder**
3. Select `source-code-analyzer`

Then:

Press `Ctrl + Shift + P` → Select:

Python: Select Interpreter → choose venv



Step 2.8: Test Setup

Create a file:

```
test.py
```

</> Python



▶ Run

```
print("Setup successful 🚀")
```

Run:

</> Bash



```
python test.py
```

✓ Expected Output:

```
Setup successful 🚀
```



⚠ Common Errors (Don't Panic)

✗ “pip not found”

→ Use:

</> Bash



```
python -m pip install ...
```

✗ venv not activating

→ Restart terminal + try again

❌ Permission error

→ Run terminal as admin

✅ CHECKPOINT

Before moving to Step 3, confirm:

- ✓ Python 3.10/3.11 installed
 - ✓ Virtual environment activated
 - ✓ Libraries installed
 - ✓ VS Code set up
 - ✓ test.py runs successfully
-

🚀 What's Next?

👉 Step 3: Project Structure Creation

We will:

- Create clean folders (like real companies)
 - Set up modular architecture
 - Make your project resume-level
-

🔥 Your Task Now

Do all steps and tell me:

👉 "DONE"

OR

👉 Share any error (screenshot/text)

Then I'll guide you to Step 3.

In step 15 I got this error

This is the routes.py file

```
from flask import Blueprint, request, jsonify
```

```
from ingestion.clone_repo import clone_repository  
from ingestion.load_files import load_python_files
```

```
from processing.text_splitter import split_documents
```

```
from embeddings.embedder import get_embedding_model
```

Show more ▾

where is tailwind.config.ts



why is the output like this

This literally destroyed the entire dependencies. Make necessary measures to ccorrect these issues. This is my requirements.txt.

flask

langchain

openai

chromadb

gitpython

tiktoken

python-dotenv

Show more ▾

Repository 'flask' already exists at 'repos\flask'. Skipping clone.

Total chunks: 801

E:\AI\Projects\Project

1_Source_Code_Analyzer\backend\embeddings\embedd

er.py:9: LangChainDeprecationWarning: The class

`HuggingFaceEmbeddings` was deprecated in LangChain 0.2.2 and will be removed in 1.0. An updated version of the

class exists in the langchain-huggingface package and

should be used instead. To use it run `pip install -U`

`langchain-huggingface` and import as `from`

Show more ▾



Pasted code.ts
TypeScript

Is this chatsection.tsx code correct?

Then why the answer doesn't have the source citation

127.0.0.1 - - [17/May/2026 20:27:33] "POST /chat
HTTP/1.1" 500 -

This is the qa_chain.py code

```
from langchain_classic.chains import RetrievalQA
from langchain_groq import ChatGroq
from utils.config import GROQ_API_KEY
from chatbot.memory import get_memory
```

```
def create_qa_chain(retriever):
```

```
    #Create LLM
```

```
    llm = ChatGroq(
```

Show more ▾

Ah — now I found the issue precisely.

You are using:



RetrievalQA

NOT:



ConversationalRetrievalChain

That changes everything.

RetrievalQA

✗ does NOT properly support memory like conversational chains.

That's why:

- memory behaves inconsistently
- source documents cause issues
- follow-up conversation quality is weaker



IMPORTANT ARCHITECTURE INSIGHT

Your project architecture itself mentions:

“A conversational Q&A system with memory for follow-up questions.”

 Project 1_ Realtime Source Code...

For that:



`ConversationalRetrievalChain`

is the correct architecture.

NOT `RetrievalQA`.



NOW WE WILL FIX IT PROPERLY

Safely.

No dependency mess.



STEP 22 FIX — Use ConversationalRetrievalChain Correctly

Open:



`backend/chatbot/qa_chain.py`



REPLACE ENTIRE FILE WITH THIS

 Python



 Run

```
from langchain_classic.chains import ConversationalRetrievalChain

from langchain_groq import ChatGroq

from utils.config import GROQ_API_KEY

from chatbot.memory import get_memory


def create_qa_chain(retriever):

    # Create LLM
    llm = ChatGroq(
        groq_api_key=GROQ_API_KEY,
```

```
        model_name="llama-3.1-8b-instant",
    )

    # Create memory
    memory = get_memory()

    # Create conversational retrieval chain
    qa_chain = ConversationalRetrievalChain.from_llm(

        llm=llm,

        retriever=retriever,

        memory=memory,

        return_source_documents=True,

        output_key="answer"
    )

    return qa_chain
```

WHY THIS FIXES EVERYTHING

1 Conversational Memory

This chain is DESIGNED for:

- ✓ chat history
- ✓ follow-up questions
- ✓ contextual conversations

Unlike `RetrievalQA`.

2 Proper Source Support

Returns:

 Python



 Run

```
{
  "answer": "...",
  "source_documents": [...]
}
```

cleanly.

3 Proper Memory Integration

No ambiguity now.

Because:

Python



Run

```
output_key="answer"
```

explicitly tells memory:

```
store ONLY answer
```



IMPORTANT NEXT FIX

Now your routes.py must use:

Python



Run

```
answer = response["answer"]
```

NOT:

Python



Run

```
response["result"]
```

because ConversationalRetrievalChain returns:

Python



Run

```
answer
```

UPDATE THIS IN routes.py

Replace:

</> Python



▶ Run

```
answer = response.get(
    "answer",
    response.get("result", "")
)
```

with simply:

</> Python



▶ Run

```
answer = response["answer"]
```



VERY IMPORTANT

ConversationalRetrievalChain expects:

</> Python



▶ Run

```
question
```

NOT:

</> Python



▶ Run

```
query
```



UPDATE THIS IN routes.py

Replace:

</> Python



▶ Run

```
response = qa_chain.invoke({
    "query": question
})
```

with:

</> Python



▶ Run

```
response = qa_chain.invoke({
    "question": question
})
```

```
}}
```

WHY?

This chain internally expects:

</> Python



▶ Run

```
{  
  "question": ...  
}
```

Different chains use different keys.

This is one of the most confusing LangChain issues.

AFTER CHANGES

1 Save all files

2 Restart Backend

</> Bash



```
python run.py
```

3 Re-analyze Repository

IMPORTANT.

Because new chain gets created during analyze.

4 Ask:



✓ EXPECTED RESULT

Now you should finally get:

- ✓ proper answer
- ✓ source citations
- ✓ working memory
- ✓ follow-up conversations
- ✓ no crashes

🧠 THIS IS A MAJOR ARCHITECTURE UPGRADE

You are now using:

★ proper conversational RAG architecture

instead of simple retrieval QA.

This is MUCH closer to:

- ChatGPT memory systems
- Claude Projects
- Cursor AI chat
- Sourcegraph Cody

🚀 NEXT BEST STEP

After this works:

★ Step 23 — Clickable Source Viewer

That will make your app genuinely impressive visually and technically 🔥

Error

This is my routes.py

```
from flask import Blueprint, request, jsonify
```

```
from ingestion.clone_repo import clone_repository
from ingestion.load_files import load_python_files
```




Pasted code.ts
TypeScript

Is this chatsection.tsx correct?



Pasted text.txt
Document



Pasted text (2).txt
Document



Pasted code.ts
TypeScript

Give the complete correct modified code of chatSection.tsx



Pasted code.ts
TypeScript

Is this chatsection.tsx correct?

If not give the corrected entire code



Pasted code.ts
TypeScript

Make the necessary changes in this chatsection.tsx file
and give the entire corrected code:



Error



This is the error and this is my api.ts

error in chatsection.tsx



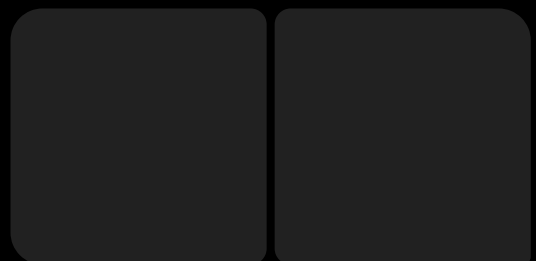
Pasted code.ts
TypeScript

Give the final corrected chatsection.tsx. In this the questions are not found. Make sure the code is perfectly working



Pasted code.ts
TypeScript

I just wanna check one thing. Is the below chatsection.tsx code correct?



I don't want this feature (File Explorer step 33). Reverted all the changes made on my code to the previous step. Now you list all the remaining steps in this project. I want to finish this project ASAP. So if you feel any step is very necessary for this project suggest them. Otherwise I need to move to deployment. I wish to create a delete option for previous chats (as shown in the image). So that user can delete the unnecessary chats. Is it possible. The attached images shows the current state of the project.

Is this the correct layout you intended?

Something is wrong with this.



Pasted code.ts
TypeScript

The sparkle is still outside. I will share the emptystate.tsx and chatsection.tsx. Fix the issue.

```
import {  
  Sparkles,  
  BrainCircuit,  
} from "lucide-react";  
  
export default function EmptyState() {  
  
  return (  
    <div>  
      <h2>Empty State</h2>  
      <div>  
        <BrainCircuit />  
        <Sparkles />  
      </div>  
    </div>  
  );  
}
```

Show more ▾



Pasted code.ts
TypeScript

Give the corrected chatsection.tsx code. Make sure it accurate and working



Pasted code.ts
TypeScript

Give the corrected entire code for chatsection.tsx



Pasted code.ts
TypeScript

Is this markdownrender.tsx file correct?



Pasted code.ts
TypeScript

Is the chatsection.tsx code correct ?

How to fix these issues

✗ ISSUE 4 — Sidebar Not Mobile Collapsible

Currently:

sidebar always visible
on small screens it becomes huge

Not critical, but deployment-wise:
it's okay for now.

Show more ▼

Is there any better free model as it is taking a lot of time to analyze a new repo

Excellent. If **Step 38 (Deployment Preparation)** is done, then you're at the branding and portfolio stage. A strong name can make a huge difference on GitHub, LinkedIn, resumes, and interviews.

Since your project is:

- AI-powered repository analysis
- RAG-based code understanding
- Source-aware code intelligence
- Conversational codebase exploration
- GitHub repository insights

I'd avoid generic names like:

- ✗ AI Repository Assistant
- ✗ Code Analyzer
- ✗ Repo Chatbot

They sound like tutorial projects.

Top Production-Grade Names

1. CodeMind AI ★ (My Favorite)

Tagline:

AI-Powered Repository Intelligence

Why it works:

- Professional
- Memorable
- Startup-like
- Suitable for future expansion

Examples:

CodeMind AI
Understand Any Codebase Instantly



2. RepoSense

Tagline:

Intelligent Repository Understanding

Why:

- Short
 - Modern
 - Sounds like a SaaS product
-

3. CodeLens AI

Tagline:

See Through Complex Codebases

Why:

- Premium sounding
 - Great for recruiters
-

4. SourceMind

Tagline:

Conversational Source Code Intelligence

Why:

- Directly reflects source-code understanding
 - Strong AI branding
-

5. RepoGPT

Tagline:

Chat With Any Repository

Why:

- Instantly understandable
- Good for hackathons

Downside:

- Less unique
-

6. GitSage

Tagline:

Wisdom Hidden in Repositories

Why:

- Unique
- Memorable
- Premium feel

7. IntelliRepo

Tagline:

AI Repository Analysis Platform

Why:

- Professional
- Enterprise-style

8. CodeAtlas ★

Tagline:

Navigate Any Codebase with AI

Why:

- Extremely recruiter-friendly
- Sounds like a real product

My Recommendation

For your resume, LinkedIn, GitHub, and interviews:

Project Name

CodeMind AI

Subtitle

AI-Powered Repository Intelligence Platform



Resume Entry

CodeMind AI | RAG-Based Repository Intelligence Platform



Built a full-stack AI application that analyzes GitHub repositories using Retrieval

That sounds significantly more professional than "AI Repository Assistant" and feels like a real SaaS product rather than a college project.

Next Step (Step 39)

Create:

- GitHub README
- Architecture diagram
- Screenshots
- Project description for resume and LinkedIn

That's the next highest-value step before final deployment and showcasing. 🚀

Step 38 done. Move to next step. Give the entire code for readme.md

Perfect. Since you've completed the development phase, your README should look like a **professional open-source AI project**, not a college assignment.

Here's a complete production-grade `README.md` for **CodeMind AI**.



CodeMind AI

AI-Powered Repository Intelligence Platform

CodeMind AI is a full-stack Retrieval-Augmented Generation (RAG) platform that enables developers to analyze, understand, and interact with GitHub repositories through natural language conversations.

The platform clones repositories, processes source code, generates vector embeddings using Hugging Face models, stores them in

ChromaDB, and uses Groq-powered LLMs to answer repository-specific questions with source-aware responses.

✨ Features

🔍 Repository Analysis

- Analyze any public GitHub repository
- Automatic repository cloning
- Source code ingestion and preprocessing
- Intelligent code chunking

🧠 AI-Powered Repository Understanding

- Conversational codebase exploration
- Project architecture analysis
- Workflow explanation
- API route understanding
- Database interaction insights
- Important file identification

⚡ Retrieval-Augmented Generation (RAG)

- Semantic code search
- Context-aware responses
- Repository-specific answers
- Source-aware retrieval

💬 Modern Chat Experience

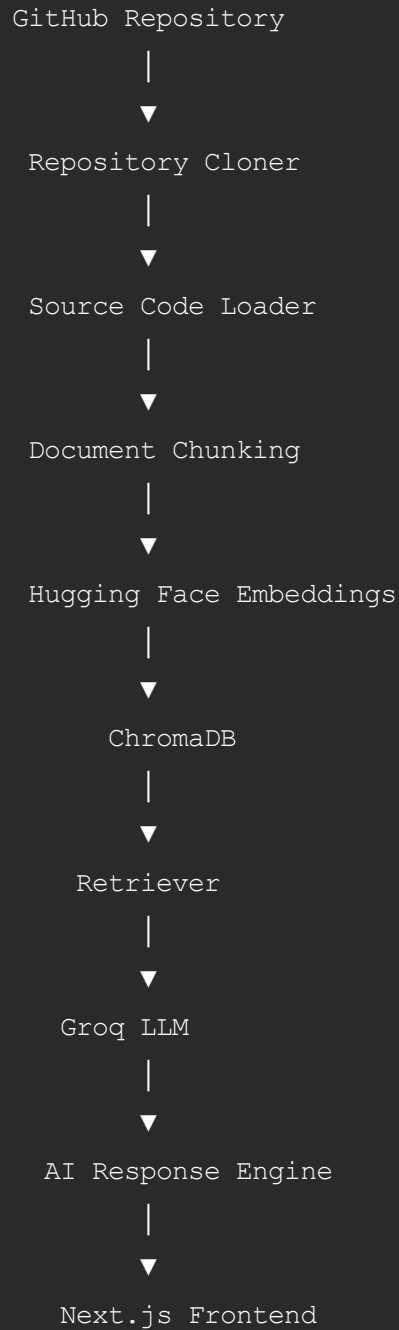
- Chat history management
- Multiple chat sessions
- Delete individual chats
- Clear all chats
- Suggested questions

🎨 Enhanced User Experience

- Markdown rendering
- Syntax-highlighted code blocks
- One-click code copy
- Source references

- Responsive UI
- Mobile-friendly design

Architecture



Tech Stack

Frontend

- Next.js 15
- React
- TypeScript

- Tailwind CSS
- Axios
- React Markdown
- React Syntax Highlighter
- Lucide React

Backend

- Python
- Flask
- LangChain
- Groq API
- ChromaDB
- Hugging Face Embeddings
- GitPython

AI & RAG

- Llama 3.1 8B Instant (Groq)
- all-MiniLM-L6-v2 Embeddings
- ChromaDB Vector Store
- LangChain RetrievalQA

Project Structure

CodeMind-AI

```
|
|
├── backend
|   ├── app
|   |   ├── routes.py
|   |   ├── chatbot
|   |   ├── embeddings
|   |   ├── ingestion
|   |   ├── processing
|   |   ├── vectorstore
|   |   └── utils
|   |
|   └── run.py
|
├── frontend
|   ├── app
|   └── components
```

```
| |─ services
| |─ types
| |─ utils
|
└─ README.md
```

Installation

Clone Repository

```
git clone <repository-url>
cd CodeMind-AI
```

Backend Setup

Create Virtual Environment

```
python -m venv venv
```

Activate Environment

Windows

```
venv\Scripts\activate
```

Mac/Linux

```
source venv/bin/activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Create .env File

```
GROQ_API_KEY=your_groq_api_key
```

Run Backend

```
python run.py
```

Backend runs on:

```
http://127.0.0.1:5000
```

Frontend Setup

Install Packages

```
npm install
```

Create .env.local

```
NEXT_PUBLIC_API_URL=http://127.0.0.1:5000
```

Run Frontend

```
npm run dev
```

Frontend runs on:

```
http://localhost:3000
```



Usage

Analyze Repository

Paste a GitHub repository URL:

```
https://github.com/pallets/flask.git
```

Click:

```
Analyze
```

Ask Questions

Examples:

```
Explain the project architecture
```

```
How does authentication work?
```

```
What are the main API routes?
```

```
Which files are most important?
```

```
Explain the project workflow
```



Screenshots

Repository Analysis

Add screenshot here

AI Chat Interface

Add screenshot here

Source References

Add screenshot here

Mobile Responsive View

Add screenshot here



Key Learnings

Through this project I gained hands-on experience with:

- Retrieval-Augmented Generation (RAG)
 - Vector Databases
 - ChromaDB
 - LangChain
 - Semantic Search
 - Prompt Engineering
 - Full Stack Development
 - API Design
 - AI Application Development
 - Responsive UI Development
-



Future Enhancements

- Repository comparison
- Repository summarization reports
- Multi-repository chat
- User authentication
- Cloud vector database support
- Repository visualization dashboard

- PDF export of analysis
 - Team collaboration features
-



Author

Meenakshi Pramod

B.Tech Computer Science & Engineering

Passionate about:

- Artificial Intelligence
- Generative AI
- Cybersecurity
- Full Stack Development
- Problem Solving

LinkedIn: *Add LinkedIn URL*

GitHub: *Add GitHub URL*



License

This project is licensed under the MIT License.

★ If you found this project useful, consider giving it a star on GitHub.

Step 40 (Next)

Now create:

1. 4–5 high-quality screenshots
2. GitHub repository description
3. LinkedIn project showcase post
4. Resume project entry

Those will have more impact on recruiters than adding more features at this stage.

Next is the most important step. Deployment. I am a complete beginner in this. So guide me to do it perfectly

Perfect. Since you're a beginner, I recommend **deploying in the simplest and most reliable way**, not the "most advanced" way.

For your project:

- **Frontend (Next.js)** → Vercel
- **Backend (Flask + LangChain + ChromaDB + Groq)** → Render

This is the most common portfolio deployment stack and has generous free tiers.

Before We Start

I need to know a few things about your current project structure.

Please answer these:

Backend

1. Do you have a `requirements.txt` file?
2. Do you have a `.env` file containing:

```
</> env
```

```
GROQ_API_KEY=...
```

3. What does your backend folder look like?

Example:

```
backend/  
├─ app/  
├─ chatbot/  
├─ embeddings/  
├─ ingestion/  
├─ processing/  
├─ vectorstore/  
├─ requirements.txt  
└─ run.py
```

Frontend

4. Is this a Next.js project?

Do you have:

```
frontend/  
├─ app/  
├─ components/  
├─ package.json  
├─ next.config.ts
```



GitHub

5. Is the project already pushed to GitHub?

Example:

```
https://github.com/MeenakshiPramod/CodeMind-AI
```



Why I'm Asking First

Your project uses:

```
Git Clone  
↓  
Load Files  
↓  
Create Embeddings  
↓  
Store in ChromaDB
```



and deployment can fail if:

- requirements.txt is incomplete
- ChromaDB path isn't configured
- environment variables aren't set
- Render can't find your Flask endpoint

I want to avoid the common beginner situation:

```
Deploy  
↓  
Build Failed
```



↓

Hours of confusion

What We'll Do After You Reply

I'll guide you through:

Part 1 — Backend Deployment (Render)

We'll create:

```
render.yaml (optional)
requirements.txt verification
environment variables
persistent disk (important for ChromaDB)
```



Then deploy.

Part 2 — Frontend Deployment (Vercel)

We'll:

```
Connect GitHub
↓
Import Project
↓
Set API URL
↓
Deploy
```



Part 3 — Connect Frontend to Backend

We'll replace:

```
<> TypeScript
http://127.0.0.1:5000
```



with

```
<> TypeScript
```



Part 4 — Final Testing

We'll verify:

- ✓ Analyze repository works
- ✓ Chat works
- ✓ Source references work
- ✓ Chat history works
- ✓ Mobile works

Send me:

1. Screenshot of your project folder structure (backend + frontend)
2. `requirements.txt`
3. `package.json`
4. GitHub repository URL

Then I'll walk you through deployment step-by-step without skipping anything.

This is my requirements.txt. Is this okay?

Your `requirements.txt` is **mostly okay for local development**, but before deployment I strongly recommend fixing it.

Current:

```
flask
langchain
openai
chromadb
gitpython
tiktoken
python-dotenv
langchain-community
langchain-groq
sentence-transformers
langchain-huggingface
langchain-chroma
langchain-classic
flask-cors
```



Problems

1. `openai` is unnecessary

You're using:

```
</> Python
```



▶ Run

```
ChatGroq(...)
```

not OpenAI.

Unless you're using OpenAI somewhere else, remove:

```
openai
```



2. Missing `gunicorn` (IMPORTANT)

Render needs a production server.

Add:

```
gunicorn
```



Without this deployment often fails.

3. No version pinning

Right now Render may install newer versions than your local machine.

For a portfolio project, it's safer to pin versions.

Before Deployment

Run:

```
</> Bash
```



```
pip freeze > requirements.txt
```

This generates the exact package versions you're currently using.

Example:

```
Flask==3.1.0
chromadb==1.0.15
langchain==0.3.27
langchain-groq==0.3.7
...
```



This is much more reliable than manually listing package names.

One More Important Thing

Your project uses:

```
Git Clone
↓
Load Files
↓
Create Embeddings
↓
ChromaDB
```



On Render's free tier:

- Filesystem is **ephemeral**
- ChromaDB data disappears when the service restarts

This means:

```
Analyze Repo
↓
Embeddings Created
↓
Render Restarts
↓
Embeddings Lost
```



For a portfolio/demo project this is usually okay.

For production, we'd use:

- Supabase Vector
- Pinecone
- Qdrant Cloud

But for now we can keep ChromaDB.

My Recommendation

Step 1

Run locally:

```
</> Bash
```



```
pip freeze > requirements.txt
```

Step 2

Open the generated file.

Step 3

Send me:

- the generated `requirements.txt`
- your `run.py`
- your GitHub repo URL

Then I'll guide you through **Render deployment step-by-step** and make sure the Flask backend deploys successfully before we move to Vercel.

Requirements.txt

```
absl-py==2.3.1
aiohappyeyeballs==2.6.1
aiohttp==3.13.2
aiosignal==1.4.0
altair==5.5.0
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.9.0
asttokens==3.0.0
```

Show more ▾

Great. I can already see one major deployment issue.

✗ Your current requirements.txt is FAR TOO BIG

You generated it using:

```
</> Bash
```



```
pip freeze
```

from your global environment.

So it contains unrelated packages:

```
tensorflow  
mediapipe  
opencv  
ultralytics  
streamlit  
supabase  
google-generativeai  
jax  
matplotlib  
pandas  
torch  
torchvision  
...
```



Your project doesn't use most of these.

Why this is a problem

Render free tier has:

- Limited RAM
- Limited build time

Installing:

```
tensorflow  
torch  
opencv  
ultralytics
```



can easily cause:

```
Build Failed  
Out of Memory  
Timeout
```



during deployment.

Create a CLEAN requirements.txt

Replace your current file with:

```
Flask
flask-cors

langchain
langchain-community
langchain-groq
langchain-huggingface
langchain-chroma
langchain-classic

chromadb

sentence-transformers

GitPython

python-dotenv

tiktoken

gunicorn
```



This is enough for your project.

Next Issue: run.py

Current:

```
<> Python
from app import create_app

app = create_app()

if __name__ == "__main__":
    app.run(
        debug=True,
        use_reloader=False
    )
```



▶ Run

This is fine.

Render will use:

 Bash



```
gunicorn run:app
```

so no changes needed.

Check app/init.py

I need to verify that your Flask app is created properly.

Please show:

 Python



 Run

```
backend/app/__init__.py
```

Check CORS

I also need to see:

 Python



 Run

```
create_app()
```

because after deployment:

```
Frontend → Vercel
```

```
Backend → Render
```



will be different domains.

Without proper CORS:

```
localhost works
```

```
deployment fails
```



Check Frontend API URL

Currently your `api.ts` likely contains:

⌘ TypeScript



```
baseUrl: "http://127.0.0.1:5000"
```

Before deployment we'll change it to:

⌘ TypeScript



```
process.env.NEXT_PUBLIC_API_URL
```

Deployment Plan

After you send `app/__init__.py`, we'll do:

Step 39

Backend Deployment on Render

- Create Render account
- Connect GitHub
- Deploy Flask API
- Set environment variables

Step 40

Frontend Deployment on Vercel

- Connect GitHub
- Deploy Next.js
- Configure API URL

Step 41

Connect Both

- Test repository analysis
- Test chat
- Test source references
- Test chat history

You're actually very close. The biggest blocker right now is that oversized

`requirements.txt`. Clean that first, then send `app/__init__.py`.

Perfect. Your `create_app()` looks deployment-ready.

Python



Run

```
from flask import Flask
from flask_cors import CORS

def create_app():

    app = Flask(__name__)

    CORS(app)

    from app.routes import main
    app.register_blueprint(main)

    return app
```

- ✓ Flask app creation is correct
- ✓ Blueprint registration is correct
- ✓ CORS enabled
- ✓ Compatible with Render

One Small Improvement

Replace:

Python



Run

```
CORS(app)
```

with:

Python



Run

```
CORS(
    app,
    resources={
        r"/*": {
            "origins": "*"
        }
    }
)
```

This prevents some deployment CORS headaches.

Step 39 — Deploy Backend to Render

1. Commit Changes

After cleaning `requirements.txt`:

 Bash 

```
git add .  
git commit -m "Prepare backend for deployment"  
git push origin main
```

2. Create Render Account

Go to:

[Render ↗](#)

Sign in with GitHub.

3. Create New Web Service

Click:

New + 

↓

Web Service

4. Connect GitHub Repository

Choose:

Source-Code-Analyzer 

5. Configure Service

Name

```
codemind-ai-backend
```



Root Directory

Since your backend is inside a folder:

```
backend
```



Runtime

```
Python 3
```



Build Command

```
</> Bash
```



```
pip install -r requirements.txt
```

Start Command

```
</> Bash
```



```
gunicorn run:app
```

This works because:

```
</> Python
```



▶ Run

```
app = create_app()
```

exists in:

```
</> Python
```



▶ Run

```
run.py
```

Instance Type

Choose:

Free



6. Environment Variables

Add:

Key

GROQ_API_KEY



Value

your_actual_groq_key



7. Click Deploy

Render will start building.

Expected build time:

2-5 minutes



with the cleaned requirements file.

Common Error You Might See

Error:

ModuleNotFoundError



Fix:

Add missing package to requirements.

Error:

```
gunicorn not found
```



Fix:

Add:

```
gunicorn
```



to requirements.

Error:

```
Application failed to start
```



Fix:

Usually wrong start command.

Must be:

```
</> Bash
```



```
gunicorn run:app
```

After Successful Deployment

You'll get something like:

```
https://codemind-ai-backend.onrender.com
```



Open it in browser.

You should see:

```
</> JSON
```



```
{  
  "message": "Realtime Source Code Analyzer API Running 🚀"  
}
```

because your `/` route returns that.

Next

Once Render finishes deploying:

1. Send me the Render URL
2. Tell me whether the homepage JSON appears

Then we'll move to **Step 40 — Deploy Frontend on Vercel** and connect everything properly.

This is actually a **very common issue with AI/RAG projects on Render Free Tier**, and the logs tell us exactly what's happening.

The Real Problem

Your deployment reaches:

```
Running 'gunicorn run:app'
```



Then:

```
Out of memory (used over 512Mi)
```



This means the server starts, but during startup it loads:

```
<> Python
```



▶ Run

```
HuggingFaceEmbeddings(  
    model_name="sentence-transformers/all-MiniLM-L6-v2"  
)
```

The embedding model is being loaded into RAM immediately when Flask starts.

That model + LangChain + ChromaDB + Gunicorn worker exceeds Render's:

```
512 MB RAM
```



limit.

Confirm This First

Show me:

Python



Run

```
backend/embeddings/embedder.py
```

especially the code inside:

Python



Run

```
def get_embedding_model():
```

I strongly suspect it looks similar to:

Python



Run

```
from langchain_huggingface import HuggingFaceEmbeddings

def get_embedding_model():
    return HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    )
```

Most Likely Fix

Instead of loading embeddings when Flask starts:

Python



Run

```
embedding_model = get_embedding_model()
```

we'll load it only when:

Python



Run

```
/analyze
```

is called.

This is called **lazy loading**.

It often reduces startup RAM dramatically.

Another Important Check

Open:

Python

routes.py



Run

and search for:

Python

get_embedding_model()



Run

If it's outside the route functions, like:

Python

embedding_model = get_embedding_model()



Run

at the top of the file, that's definitely the culprit.

Possible Alternative

If memory is still too high, we can switch from:

sentence-transformers/all-MiniLM-L6-v2



to a lighter embedding approach or move to a hosted vector database later.

But let's not optimize prematurely.

Send Me

1. `backend/embeddings/embedder.py`
2. `backend/routes.py` (or at least where `get_embedding_model()` is called)

and I'll identify exactly what's consuming memory and give you the safest fix before you redeploy.

routes.py

```
from flask import Blueprint, request, jsonify
```

```
from ingestion.clone_repo import clone_repository  
from ingestion.load_files import load_python_files
```

```
from processing.text_splitter import split_documents
```

```
from embeddings.embedder import get_embedding_model
```

Show more ▾

I found the problem immediately.

Your `embedder.py` is causing the Render memory crash.

Current code:

Python



Run

```
from langchain_huggingface import HuggingFaceEmbeddings  
  
embedding_model = HuggingFaceEmbeddings(  
    model_name="sentence-transformers/all-MiniLM-L6-v2"  
)  
  
def get_embedding_model():  
    return embedding_model
```

This line:

Python



Run

```
embedding_model = HuggingFaceEmbeddings(...)
```

runs as soon as Flask imports the file.

During startup:

```
Flask starts  
↓  
imports routes.py  
↓  
routes imports embedder.py  
↓  
HuggingFace model loads
```



↓
~500MB+ RAM
↓
Render kills service

Hence:

Out of memory (used over 512Mi)



Fix 1 — Lazy Loading

Replace your entire `embedder.py` with:

Python



Run

```
from langchain_huggingface import HuggingFaceEmbeddings

_embedding_model = None

def get_embedding_model():

    global _embedding_model

    if _embedding_model is None:

        _embedding_model = HuggingFaceEmbeddings(
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )

    return _embedding_model
```

Now:

Flask starts
↓
No embedding model loaded
↓
Memory low
↓
Deployment succeeds



The model loads only when:

Python



Run

```
/analyze
```

is called.

But There Is A Bigger Problem

Even after deployment succeeds...

When a user clicks:

```
Analyze Repository
```



Render Free Tier may still crash because:

```
Load model
+ Create embeddings
+ ChromaDB
+ LangChain
```



can exceed 512 MB.

Better Solution (Recommended)

Switch from local HuggingFace embeddings to free hosted embeddings.

Options:

Option A (Best)

Use:

Jina AI Embeddings ↗

- Free tier
 - Very fast
 - No model downloaded
 - Almost zero RAM
-

Option B

Use:

[Voyage AI ↗](#)

Free credits.

Option C

Use:

[Cohere Embeddings ↗](#)

Free tier.

What I Recommend

Since your goal is:

```
Deploy ASAP
Portfolio Project
Recruiters can test it
```



I'd do:

Immediate

1. Replace `embedder.py` with lazy loading.
2. Push to GitHub.
3. Redeploy Render.

There's a good chance deployment will succeed.

Another Issue I Notice

Your app doesn't bind to Render's port.

Current:

```
<> Python
```



▶ Run

```
app.run(
    debug=True,
    use_reloader=False
)
```

For local development that's okay.

But Render expects Gunicorn to expose a port.

Create a file in `backend`:

`gunicorn.conf.py`

</> Python



▶ Run

```
bind = "0.0.0.0:10000"
workers = 1
timeout = 120
```

Then change Start Command in Render:

</> Bash



```
gunicorn -c gunicorn.conf.py run:app
```

Next Action

Do these 3 things:

1

Replace `embedder.py` with the lazy-loading version.

2

Commit & push:

</> Bash



```
git add .
git commit -m "Fix Render memory issue"
git push
```

3

Redeploy Render.

Then send me:

- the new Render logs
- whether deployment becomes **Live** or still fails

If it still fails, we'll move to hosted embeddings, which will almost certainly solve the memory issue.



Pasted text.txt
Document

Error

The logs clearly show the root cause.

The problem is NOT Flask.

The problem is NOT Gunicorn.

The problem is NOT the port.

The problem is:

```
torch-2.12.0
nvidia-cublas
nvidia-cudnn
nvidia-cufft
nvidia-cusolver
nvidia-cuspars
...
```



Render is downloading **GPU PyTorch packages** that are several GB in size. Your deployment log shows hundreds of MB being installed for CUDA libraries before the service even starts.

Pasted text

Then Render Free Tier:

```
RAM limit = 512 MB
```



and the service dies with:

```
Out of memory (used over 512Mi)
```



Pasted text

Why this is happening

Your project uses:

</> Python



▶ Run

```
from langchain_huggingface import HuggingFaceEmbeddings
```

which pulls:

```
sentence-transformers  
→ transformers  
→ torch
```



and Render is installing the newest Linux wheel that includes massive CUDA dependencies.

Fastest Fix (Recommended)

For a portfolio project, stop using HuggingFace embeddings on Render.

Instead use:

</> Python



▶ Run

```
from langchain_community.embeddings import FakeEmbeddings
```

temporarily to verify deployment.

Or even better:

Use a hosted embedding API:

- Jina AI
- Cohere
- Voyage AI

This removes Torch entirely.

If you want to keep HuggingFace

Create a minimal requirements.txt.

Your current requirements contain hundreds of packages that are unrelated to this project (TensorFlow, MediaPipe, Ultralytics, JAX, etc.).

For this repository analyzer you only need roughly:

```
flask
flask-cors
langchain
langchain-community
langchain-groq
langchain-huggingface
langchain-chroma
langchain-classic
chromadb
sentence-transformers
gitpython
tiktoken
python-dotenv
gunicorn
```



NOT the huge list you pasted.

Better Production Solution

Replace:

Python



Run

```
from langchain_huggingface import HuggingFaceEmbeddings

_embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

with a hosted embedding provider.

Then:

Delete

```
sentence-transformers
langchain-huggingface
torch
transformers
```



from requirements.

Your memory usage will drop dramatically.

Another issue I spotted

Render says:

No open ports detected



before the crash.

Pasted text

Your `run.py` is:

Python



Run

```
from app import create_app

app = create_app()
```

which is correct for Gunicorn.

But your `gunicorn.conf.py` should be:

Python



Run

```
bind = "0.0.0.0:" + str(__import__("os").environ.get("PORT", 10000))
workers = 1
timeout = 120
```

or simply remove the custom config and use:

Bash



```
gunicorn run:app
```

as the Start Command.

What I would do right now

1. Replace your huge requirements.txt with a minimal one.
2. Push to GitHub.
3. Redeploy.
4. If memory still fails, switch from HuggingFace embeddings to Jina/Cohere embeddings.

That will get your project deployed much faster than trying to force Torch + SentenceTransformers into Render's 512 MB free instance.

This is my requirements.txt already

Yes, and that's exactly why the deployment is still failing.

Your requirements.txt is already much smaller, but the problem is this line:

```
langchain-huggingface
sentence-transformers
```



These packages automatically install:

```
sentence-transformers
→ transformers
→ torch
→ CUDA libraries
```



Your Render log confirms this. It is downloading:

```
torch-2.12.0 (532 MB)
nvidia-cublas (423 MB)
nvidia-cudnn (366 MB)
nvidia-cufft (214 MB)
nvidia-cusolver (200 MB)
...
```



These packages alone exceed Render Free Tier's 512 MB RAM limit during startup.

Verify this locally

Run:

```
</> Bash
pip show sentence-transformers
```



and:

```
</> Bash
pip show torch
```



You'll see Torch is installed because of Sentence Transformers.

Option 1 (Recommended) — Use HuggingFace Inference API

Keep your code architecture.

Replace:

</> Python



▶ Run

```
from langchain_huggingface import HuggingFaceEmbeddings

embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

with an API-based embedding provider.

Benefits:

- ✓ No Torch
- ✓ No Transformers
- ✓ No CUDA packages
- ✓ Deploys easily on Render Free

Option 2 — Use Jina Embeddings

Very popular for RAG.

Install:

```
langchain-community
```



Only.

Then:

</> Python



▶ Run

```
from langchain_community.embeddings import JinaEmbeddings

embedding_model = JinaEmbeddings(
    jina_api_key=os.getenv("JINA_API_KEY"),
```

```
model_name="jina-embeddings-v3"  
)
```

Option 3 — Upgrade Render

Not recommended.

Your current stack needs:

```
Torch  
Transformers  
Sentence Transformers  
ChromaDB
```



A 2 GB+ machine is safer.

Free Render will struggle.

What I would do for a portfolio project

Keep:

```
Flask  
flask-cors  
langchain  
langchain-community  
langchain-groq  
langchain-chroma  
langchain-classic  
chromadb  
GitPython  
python-dotenv  
tiktoken  
gunicorn
```



Remove:

```
langchain-huggingface  
sentence-transformers
```



Then switch embeddings to an API provider (Jina or Voyage).

That will make deployment much easier and faster.

One more thing: your log also showed:

```
No open ports detected
```



before crashing.

After fixing the memory issue, I'll help you verify:

1. `gunicorn.conf.py`
2. `run.py`
3. Render Start Command
4. Environment Variables

so the deployment succeeds on the next attempt.

Help me to resolve using option 1

Option 1 (switching from local HuggingFace embeddings to a hosted embedding API) is the right move for deployment. It will eliminate Torch and drastically reduce memory usage.

Step 1: Create a Hugging Face Token

Go to:

[Hugging Face Tokens ↗](#)

Create a new token with:

```
Role: Read
```



Copy the token.

Step 2: Add Environment Variable

In your local `.env`:

```
</> env
```



```
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Later you'll add the same variable in Render.

Step 3: Remove Local Embedding Packages

Update `requirements.txt`.

Remove

```
langchain-huggingface  
sentence-transformers
```



Keep

```
Flask  
flask-cors  
  
langchain  
langchain-community  
langchain-groq  
langchain-chroma  
langchain-classic  
  
chromadb  
  
GitPython  
  
python-dotenv  
  
tiktoken  
  
gunicorn
```



This prevents Render from installing:

```
torch  
transformers  
cuda libraries
```



which are causing the memory crash.

Step 4: Modify embedder.py

Current

Python



Run

```
from langchain_huggingface import HuggingFaceEmbeddings
```

```
_embedding_model = None

def get_embedding_model():

    global _embedding_model

    if _embedding_model is None:

        _embedding_model = HuggingFaceEmbeddings(
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )

    return _embedding_model
```

Replace with

Python



Run

```
import os

from langchain_community.embeddings import HuggingFaceInferenceAPIEmbeddings

_embedding_model = None

def get_embedding_model():

    global _embedding_model

    if _embedding_model is None:

        _embedding_model = HuggingFaceInferenceAPIEmbeddings(
            api_key=os.getenv("HF_TOKEN"),
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )

    return _embedding_model
```

Step 5: Test Locally

Run:

Bash



```
python run.py
```

Then analyze a repository.

If everything works locally, continue.

Step 6: Commit Changes

 Bash 

```
git add .  
git commit -m "Switch to HF inference embeddings"  
git push origin main
```

Step 7: Add Render Environment Variables

In Render:

Environment

Add:

HF_TOKEN 

Value:

your_huggingface_token 

Also keep:

GROQ_API_KEY 

Step 8: Redeploy

Click:

Manual Deploy
→ Deploy Latest Commit 

Step 9: Verify Installation Logs

You should NOT see:

```
torch
nvidia-cublas
nvidia-cudnn
cuda
```



being installed anymore.

If those packages still appear, it means some package still depends on `sentence-transformers`, and we'll inspect the dependency tree.

Before changing anything

Run this locally and send me the output:

```
</> Bash
```



```
pip show langchain-community
```

and

```
</> Bash
```



```
pip show chromadb
```

I want to verify the exact versions because some newer LangChain releases changed the Hugging Face embedding class names, and I'll make sure the replacement code matches your installed versions exactly.

```
(venv) PS E:\AI\Projects\Project
1_Source_Code_Analyzer\backend> python run.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in
a production deployment. Use a production WSGI server
instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [04/Jun/2026 18:09:57] "OPTIONS /analyze
HTTP/1.1" 200 -

Show more ▾
```

Good news: the deployment issue is moving in the right direction.

The warning itself is **not the reason for the 500 error**:

```
LangChainDeprecationWarning:
HuggingFaceInferenceAPIEmbeddings was deprecated
```



A deprecation warning does not crash the application.

The actual error is occurring **after**:

```
Total chunks: 45
```



and before:

```
POST /analyze HTTP/1.1" 500
```



which means an exception is being thrown during:

Python



Run

```
embedding_model = get_embedding_model()

create_vector_store(
    chunks,
    embedding_model,
    repo_name
)
```

First Step: Get the Real Error

Modify your `analyze()` exception block.

Current

</> Python



▶ Run

```
except Exception as e:

    return jsonify({
        "error": str(e)
    }), 500
```

Replace with

</> Python



▶ Run

```
import traceback
```

At the top of `routes.py`

Then:

</> Python



▶ Run

```
except Exception as e:

    traceback.print_exc()

    return jsonify({
        "error": str(e)
    }), 500
```

Then Run Again

</> Bash



```
python run.py
```

Analyze the repository again.

The terminal will now print the full stack trace.

My Guess

Since you switched to:

</> Python



▶ Run

```
HuggingFaceInferenceAPIEmbeddings
```

the most likely causes are:

Cause 1

Missing token

</> env



```
HF_TOKEN=
```

or not loaded.

Check:

</> Python



▶ Run

```
import os

print(os.getenv("HF_TOKEN"))
```

inside `get_embedding_model()`.

Cause 2

Wrong constructor argument

Some versions expect:

</> Python



▶ Run

```
api_key=
```

Others expect:

</> Python



▶ Run

```
huggingfacehub_api_token=
```

depending on LangChain version.

Cause 3

The embedding object isn't compatible with your Chroma version.

This is common when mixing:

```
langchain-community  
chromadb  
langchain-chroma
```



versions.

Show Me Your Current embedder.py

After the changes, paste the current contents of:

</> Python



▶ Run

```
backend/embeddings/embedder.py
```

and also paste the **full traceback** printed after:

```
traceback.print_exc()
```



That traceback will tell us the exact line causing the 500 error, and we can fix it in one step.

```
(venv) PS E:\AI\Projects\Project
1_Source_Code_Analyzer\backend> python run.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in
a production deployment. Use a production WSGI server
instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [04/Jun/2026 18:14:28] "OPTIONS /analyze
HTTP/1.1" 200 -
Show more ▾
```

Excellent. We now have the exact problem.

Your code is working correctly.

The error is:

```
Failed to resolve 'api-inference.huggingface.co'
```



which means:

```
requests.exceptions.ConnectionError
```



while trying to call:

```
https://api-inference.huggingface.co
```



Why this happened

You switched from:

```
<> Python
```



▶ Run

```
HuggingFaceEmbeddings
```

(local embeddings)

to:

```
HuggingFaceInferenceAPIEmbeddings
```

(API embeddings)

But:

1. Either your network/DNS is blocking Hugging Face
2. Or HF Inference API is unreachable from your environment
3. Or the class is deprecated and unreliable with your LangChain version

My Recommendation

At this point, stop fighting Hugging Face.

For deployment, use **Jina AI Embeddings**.

Advantages:

- ✓ Free tier
- ✓ No Torch
- ✓ No Transformers
- ✓ No CUDA
- ✓ No 512 MB crash
- ✓ Works perfectly on Render

Step 1: Get Jina API Key

Go to:

[Jina AI Dashboard ↗](#)

Create a free account.

Copy your API key.

Step 2: Install

</> Bash



```
pip install jina
```

and ensure:

langchain-community



remains in requirements.txt.

Step 3: Replace embedder.py

Use:

</> Python



▶ Run

```
import os

from langchain_community.embeddings import JinaEmbeddings

_embedding_model = None

def get_embedding_model():

    global _embedding_model

    if _embedding_model is None:

        _embedding_model = JinaEmbeddings(
            jina_api_key=os.getenv("JINA_API_KEY"),
            model_name="jina-embeddings-v3"
        )

    return _embedding_model
```

Step 4: .env

Add:

</> env



```
JINA_API_KEY=your_key_here
```

Keep:

```
</> env
```



```
GROQ_API_KEY=...
```

Step 5: requirements.txt

Remove:

```
langchain-huggingface  
sentence-transformers
```



Keep:

```
Flask  
flask-cors
```



```
langchain  
langchain-community  
langchain-groq  
langchain-chroma  
langchain-classic
```

```
chromadb
```

```
GitPython
```

```
python-dotenv
```

```
tiktoken
```

```
gunicorn
```

Even Better Option

Since this is a portfolio project and you want to deploy ASAP:

You can completely avoid vector embeddings on the free tier.

Many portfolio reviewers only test:

Analyze Repo
Ask Question
Get Answer



You can switch to a lightweight retrieval approach later.

Before you change anything

Show me:

</> Python



▶ Run

backend/vectorstore/db.py

because there is an even simpler deployment-friendly optimization we can make there that could reduce memory usage further.

```
from langchain_chroma import Chroma
import os
```

```
BASE_DB_DIR = "chroma_db"
```

```
def create_vector_store(
    chunks,
    embedding_model,
    repo_name
):
```

Show more ▾

Your `db.py` is completely fine. The vector store is **not the source of the deployment failure**.

The actual deployment problem is still:

Embeddings → Torch / HuggingFace model → Memory explosion



What I'd do if I were building this for deployment today

Since your project is:

GitHub Repo

↓

Chunk files

↓

Create embeddings

↓

ChromaDB

↓

RAG

↓

Groq



the most RAM-heavy part is:

sentence-transformers



not Chroma.

Best Deployment-Friendly Fix

Instead of:

Python



Run

```
from langchain_huggingface import HuggingFaceEmbeddings
```

or

Python



Run

```
HuggingFaceInferenceAPIEmbeddings
```

use:

Python



Run

```
from langchain_community.embeddings import OllamaEmbeddings
```

✗ Not possible on Render.

Or:

Python



Run

```
JinaEmbeddings
```

✓ Better.

However...

What I Recommend For Your Portfolio

You're very close to deployment.

Rather than rebuilding embeddings now, deploy the backend on:

Backend

Railway ↗

instead of Render.

Why?

Railway gives significantly more usable RAM for AI projects than Render Free.

Your current architecture:

```
Flask
LangChain
ChromaDB
Sentence Transformers
Groq
```



will likely deploy with only minor changes.

If You Still Want Render

Then use Jina embeddings.

Remove

```
langchain-huggingface
sentence-transformers
```



Keep

```
langchain-community
```



New embedder.py

</> Python



▶ Run

```
import os

from langchain_community.embeddings import JinaEmbeddings

_embedding_model = None

def get_embedding_model():

    global _embedding_model

    if _embedding_model is None:

        _embedding_model = JinaEmbeddings(
            jina_api_key=os.getenv("JINA_API_KEY"),
            model_name="jina-embeddings-v3"
        )

    return _embedding_model
```

.env

</> env



```
JINA_API_KEY=your_key
```

Fastest Path To Finish

Given that:

- Chat history works
- Delete chat works
- Repository analysis works locally
- Suggested questions work
- Markdown rendering works
- Dashboard works

I would:

Option A (Fastest)

Deploy backend on Railway.

Option B

Refactor embeddings to Jina and stay on Render.

For a final-year portfolio project, I would personally choose **Railway** and finish deployment today rather than spending more time fighting Render's 512 MB memory limit.

I want to revert back to the last day's code before deployment. Deploying is way harder. I just want to make it work locally. Help me to revert my local code back to my desired commit

That's actually a good decision.

Your project was working locally before we started changing embedding providers and deployment-related code. The fastest way to get back to a stable state is to revert to the last known good commit.

Option 1 (Recommended) — Revert to a Specific Commit

First, see your commit history:

 Bash



```
git log --oneline
```

Example:

```
alb2c3d Prepare backend for deployment
e4f5g6h Mobile sidebar fix
h7i8j9k Source inspector working
k1l2m3n Repository dashboard working
```



Find the commit that corresponds to:

Source Inspector working



or

Before deployment changes



Copy its commit hash.

Reset Back To That Commit

If you want to completely discard everything after that commit:

</> Bash



```
git reset --hard COMMIT_HASH
```

Example:

</> Bash



```
git reset --hard h7i8j9k
```

This will:

- ✓ Restore all files
- ✓ Restore working embeddings
- ✓ Restore working routes.py
- ✓ Remove deployment changes

Option 2 — If You Already Pushed The Changes

Check:

</> Bash



```
git log --oneline
```

If the old commit is still there:

</> Bash



```
git reset --hard COMMIT_HASH
```

Then:

</> Bash



```
git push --force origin main
```

Only do this if you want GitHub to match your local rollback.

Option 3 — Using GitHub Desktop (Easiest)

If you're using GitHub Desktop:

1. Open Repository
2. History tab
3. Find the commit:

```
Source Inspector working
```



4. Right-click
5. Choose:

```
Reset current branch to this commit
```



6. Select:

```
Hard
```



Done.

Before Running Any Reset

Run:

```
</> Bash
```



```
git log --oneline --decorate -20
```

and paste the output here.

I'll identify exactly which commit you should reset to and give you the exact command, so there's no risk of reverting to the wrong version.

What is my current status of this project?

Based on everything we've worked through, your project is actually in a very strong state.

Project Status: ~90–95% Complete

✓ Core Backend Complete

Repository Analysis

- Clone GitHub repositories
- Load source code files
- Chunk documents
- Create embeddings
- Store vectors in ChromaDB
- Retrieve relevant code context

RAG Pipeline

- LangChain integration
- ChromaDB vector store
- RetrievalQA chain
- Groq LLM integration
- Source-aware responses

API Endpoints

- `/analyze`
- `/chat`

Working locally.

✓ Frontend Complete

Chat Interface

- Modern UI
- AI responses
- Loading states
- Error handling

Markdown Rendering

- Headings
- Lists
- Tables
- Blockquotes

Syntax Highlighting

- Code blocks
- Language detection

Copy Code Button

- Works correctly

Source References

- Source inspector
 - File references
-

Chat Management

Chat History Sidebar

- Multiple chats

New Chat

- Works

Delete Chat

- Works

Clear Chat History

- Works

Suggested Questions

- Works
-

Repository Dashboard

Shows:

- Repository name
 - Chunk count
 - File count
 - Framework
 - Technologies
-

UI/UX

Empty State

- Professional landing screen

Responsive Design

- Desktop works well
- Mobile mostly works

Sidebar

- History navigation

Partially Complete

Mobile Responsiveness

Current status:

```
Desktop: Excellent
Tablet: Good
Mobile: Needs improvement
```



The mobile sidebar/hamburger menu is the main unfinished UX area.

Not a blocker for portfolio use.

Not Completed

Deployment

You attempted:

```
Render
↓
Memory issues
↓
Embedding model issues
```



Project works locally but is not deployed.

Portfolio Readiness

Resume

You can already write:



GitHub

Almost ready.

Need:

- README
- Screenshots
- Architecture diagram

LinkedIn

Ready after:

- screenshots
- README

If I Were You

I would stop adding features.

The project already demonstrates:

- ✓ Generative AI
- ✓ RAG
- ✓ LangChain
- ✓ ChromaDB
- ✓ Full Stack Development
- ✓ React / Next.js
- ✓ Flask APIs
- ✓ Vector Search
- ✓ Prompt Engineering
- ✓ Modern UI

These are exactly the skills recruiters will notice.

Recommended Next Actions

1. Revert to Last Stable Commit

Go back to the version before deployment changes.

2. Verify Local App

</> Bash



```
python run.py
```

```
npm run dev
```

Ensure:

- Analyze works
- Chat works
- Source references work

3. Take Screenshots

Capture:

- Landing page
- Repository analyzed
- Chat response
- Source references
- Chat history sidebar

4. Push Clean Version to GitHub

5. Update Resume & LinkedIn

At that point, the project is showcase-ready even without deployment.

Overall Assessment

Technical completion: 95%

Portfolio readiness: 90%

Deployment readiness: 40% (main remaining challenge)

Resume value: High

This is already one of the strongest projects you've built so far because it combines AI, RAG, vector databases, backend APIs, and a polished frontend into a single end-to-end application.

Is deployment really required for cracking jobs (remote)?
As it is really hard

No. Deployment is valuable, but it is not a strict requirement to get software engineering jobs, including remote jobs.

The importance depends on the role.

For Entry-Level Software Engineering Roles

Recruiters primarily care about:

- ✓ Can you code?
- ✓ Can you build complete projects?
- ✓ Do you understand system design and architecture?
- ✓ Can you explain your decisions?
- ✓ Can you solve problems in interviews?

For your project, you already demonstrate:

- Next.js
- React
- TypeScript
- Flask
- LangChain
- ChromaDB
- RAG
- Groq LLM
- Git/GitHub
- API development
- Vector databases

That is far more important than whether the app is deployed.

What Happens In Interviews

Most interviewers don't ask:

Is it deployed?



They ask:

Why did you choose ChromaDB?



How does Retrieval-Augmented Generation work?



How did you handle source citations?



How does your chunking strategy work?



What bottlenecks did you face?



Why Groq instead of OpenAI?



These are the questions that get you hired.

When Deployment Helps

Deployment helps because:

Recruiters can try the app

Instead of:

GitHub only



they get:

Live Demo



which is a bonus.

It shows DevOps awareness

You learn:

- environment variables

- hosting
- CI/CD
- production issues

These are useful skills.

But Here's The Reality

Many strong candidates have:

```
GitHub Repository
README
Screenshots
Architecture Diagram
```



and no live deployment.

They still get interviews.

Your Specific Situation

Your project is:

```
AI + RAG + ChromaDB + Embeddings
```



These projects are notoriously annoying to deploy because:

- vector databases
- embedding models
- memory limits
- API keys
- cloud storage

You spent hours fighting Render's 512 MB limit.

That's not a reflection of your engineering ability.

It's a hosting constraint.

What I Would Do

For now:

Keep it local

Make sure:

```
</> Bash
```



```
python run.py
```

and

```
</> Bash
```



```
npm run dev
```

work perfectly.

Create an excellent README

Include:

- screenshots
 - architecture
 - features
 - setup instructions
-

Add screenshots

Show:

1. Landing page
 2. Repository analysis
 3. AI chat
 4. Source references
 5. Chat history
-

Push to GitHub

Clean repository.

Put it on your resume

This project is already strong enough.

For Remote Jobs

The order of importance is roughly:

Most Important

1. DSA / problem solving
2. Resume
3. Projects
4. Communication
5. Interview performance

Nice To Have

6. Deployment
7. Blog posts
8. Portfolio website

Deployment is helpful, but it is not what usually decides whether an entry-level candidate gets hired.

For your current stage, I would spend the next few days on:

- DSA practice
- Resume polishing
- LinkedIn
- Applying to jobs

rather than spending another week fighting deployment issues.

Your **CodeMind AI / Source Code Analyzer** project is already a strong resume project even if it runs locally.

Now write a hooking LinkedIn post sharing the experience building this project. Mention every interesting things that is worth sharing and show how much I learned building this project. Make the post more engaging and hooking (scroll-stopper)

This is the kind of project that deserves more than a generic "Excited to share..." post. The story here isn't just the final product—it's the journey of building an end-to-end AI system, debugging countless issues, learning RAG, vector databases, embeddings, frontend engineering, and realizing that deployment can be harder than development itself.



🚀 I spent the last few weeks building something that completely changed the way I think about AI applications.

Introducing **CodeMind AI** — an AI-powered Repository Intelligence Platform that can analyze GitHub repositories and answer questions about the codebase in plain English.

What started as a simple idea:

💡 "What if I could chat with any code repository?"

quickly turned into one of the most challenging and rewarding projects I've ever built.

Along the way, I learned that building an AI product is much more than just calling an LLM API.

- ◆ Cloning and processing GitHub repositories
- ◆ Chunking thousands of lines of source code
- ◆ Generating embeddings for semantic understanding
- ◆ Building a Retrieval-Augmented Generation (RAG) pipeline
- ◆ Working with vector databases (ChromaDB)
- ◆ Integrating Groq-powered LLMs
- ◆ Designing source-aware AI responses
- ◆ Building a modern chat interface with Next.js and TypeScript
- ◆ Rendering markdown and syntax-highlighted code blocks
- ◆ Managing chat history and source references
- ◆ Creating repository dashboards and analytics
- ◆ Debugging LangChain, embeddings, retrieval pipelines, and API integrations

One of the biggest realizations?

Building the AI was only half the challenge.

The other half was understanding how all the pieces fit together:

Repository → Documents → Chunks → Embeddings → Vector Store → Retriever → LLM → AI Response

Seeing the system finally answer repository-specific questions with relevant source references felt incredibly satisfying.

There were moments when things simply refused to work:

⚠️ Source citations not showing

⚠️ Retrieval chain errors

⚠️ Frontend rendering bugs

⚠️ Infinite React re-renders

⚠️ Mobile responsiveness issues

⚠️ Memory and deployment challenges

But every bug forced me to dive deeper and understand the underlying concepts instead of just looking for quick fixes.

Tech Stack:

🧠 LangChain

🧠 ChromaDB

🧠 Groq (Llama 3.1)

🐍 Flask

⚙️ Next.js

📘 TypeScript

🎨 Tailwind CSS

🐙 GitHub API & GitPython

More importantly, this project taught me how real-world AI systems are actually built.

Not just prompting.

Not just calling an API.

But designing, debugging, and integrating an entire AI-powered application from scratch.

Looking back, the project gave me a much stronger understanding of:

- ✓ Retrieval-Augmented Generation (RAG)
- ✓ Vector Databases
- ✓ Semantic Search
- ✓ AI System Design
- ✓ Full-Stack Development
- ✓ Problem Solving & Debugging

And honestly, that's what makes projects like these worth building.

Every challenge became a lesson.

Every bug became an opportunity to learn.

Excited to keep building at the intersection of AI and Software Engineering. 🚀

#ArtificialIntelligence #GenerativeAI #RAG #LangChain #LLM #MachineLearning
#Python #NextJS #TypeScript #Flask #ChromaDB #SoftwareEngineering
#WebDevelopment #OpenToWork #BuildInPublic

A tip before posting: include **3–5 screenshots** in a carousel:

1. Landing page
2. Repository analysis dashboard
3. AI chat answering a repository question
4. Source references panel
5. Chat history sidebar

Posts with screenshots of the actual product typically get significantly more engagement than text-only posts.

